PS1 Python 作业报告

1.Flowchart：三个数的排序

代码及结果：

[1]:
```
#1、Flowchart
def Print_values(a, b, c):
    if a > b:
        if b > c:
            print(a, b, c)
        elif a > c:
            print(a, c, b)
        else:
            print(c, a, b)
    else:
        if a > c:
            print(b, a, c)
        elif b > c:
            print(b, c, a)
        else:
            print(c, b, a)
Print_values(9, 6, 3)
Print_values(3, 9, 6)
Print_values(3, 6, 9)
Print_values(6, 9, 3)
```

```
9 6 3
9 6 3
9 6 3
9 6 3
```

(1)代码要做什么

Print_values(a, b, c) 接收三个数，通过比较大小决定输出顺序。目标是让最大的数出现在最前面，最小的数出现在最后。函数直接使用 print 输出，没有用 return 返回一个排序列表。

(2)判断过程

先比较 a 和 b。如果 a > b，再看 b 是否大于 c；如果成立，顺序就是 a、b、c。如果 b 不大于 c，就继续比较 a 和 c，决定 c 应该放在中间还是最前面。如果 a 不大于 b，就进入外层 else。此时再比较 a 和 c：若 a > c，输出 b、a、c；否则再比较 b 和 c，分别输出 b、c、a 或 c、b、a。每一条分支都把三个数安排到对应位置。

```
if a > b:
    if b > c:
        print(a, b, c)
    elif a > c:
        print(a, c, b)
    else:
        print(c, a, b)
```

(3)保存的运行结果

代码测试了 (9, 6, 3)、(3, 9, 6)、(3, 6, 9) 和 (6, 9, 3)。四次都输出 9 6 3，说明这四种输入顺序得到了相同的降序结果。原来的四次测试没有覆盖所有排列，也没有测试相等的数。本次额外检查了由 0、1、2 组成的全部 27 个三元组，输出均与降序结果一致。这个检查覆盖了重复值，但不等于验证所有输入类型。

2. Matrix multiplication：矩阵乘法

代码：

```
[2]: #2. Matrix multiplication
import random

M1 = []
M2 = []

for i in range(5):
    row = []
    for j in range(10):
        row.append(random.randint(0, 50))
    M1.append(row)

for i in range(10):
    row = []
    for j in range(5):
        row.append(random.randint(0, 50))
    M2.append(row)

print("M1:")
for row in M1:
    print(row)

print("M2:")
for row in M2:
    print(row)

def Matrix_multip(M1, M2):
    result = []

    for i in range(5):
        row = []
        for j in range(5):
            total = 0
            for k in range(10):
                total = total + M1[i][k] * M2[k][j]
            row.append(total)
        result.append(row)
    return result
print("M1 * M2:")
for row in Matrix_multip(M1, M2):
```

输出结果：

```
M1:
[35, 42, 19, 40, 18, 3, 36, 9, 50, 8]
[3, 20, 16, 13, 25, 32, 6, 12, 48, 0]
[42, 10, 33, 2, 11, 28, 37, 29, 2, 8]
[18, 3, 36, 3, 39, 22, 45, 25, 19, 43]
[23, 41, 5, 39, 29, 37, 41, 3, 0, 27]
M2:
[8, 16, 2, 35, 34]
[18, 15, 46, 48, 42]
[6, 36, 16, 3, 19]
[6, 2, 6, 14, 11]
[7, 32, 12, 17, 8]
[42, 9, 27, 49, 41]
[33, 10, 24, 46, 15]
[41, 1, 2, 24, 35]
[40, 15, 20, 1, 42]
[25, 27, 32, 11, 11]
M1 * M2:
[5399, 3892, 4981, 6321, 7065]
[4687, 2830, 3552, 3900, 5427]
[4669, 3263, 3214, 6124, 5475]
[5974, 5002, 4716, 5827, 5490]
[5094, 3644, 5447, 7895, 5794]
```

主要方法生成了一个 5 × 10 的矩阵 M1 和一个 10 × 5 的矩阵 M2。每个元素用 random.randint(0, 50) 生成，包括 0 和 50。矩阵用嵌套列表保存：外层列表存放各行，内层列表存放一行中的元素。乘法部分使用 for 循环、乘法和加法完成，没有调用现成矩阵乘法函数。

(1)为什么结果是 5 × 5

矩阵乘法要求前一个矩阵的列数等于后一个矩阵的行数。这里两个数量都是 10，因此可以相乘。乘积保留 M1 的行数和 M2 的列数，所以结果有 5 行、5 列。结果中第 i 行、第 j 列的元素，是 M1 的第 i 行与 M2 的第 j 列对应元素相乘后相加。代码里的下标从 0 开始，所以 M1[0][0] 表示第 1 行第 1 列。

```
for i in range(5):
    row = []
    for j in range(5):
        total = 0
        for k in range(10):
            total = total + M1[i][k] * M2[k][j]
        row.append(total)
    result.append(row)
```

i 选择结果的行，j 选择结果的列，k 遍历需要相乘的 10 对元素。total 必须在每个新元素开始计算时归零，否则上一个元素的结果会被带入。row 在一行开始时设为空列表，用来收集这一行的 5 个结果。

(2)结果与核对

```
[5399, 3892, 4981, 6321, 7065]
[4687, 2830, 3552, 3900, 5427]
[4669, 3263, 3214, 6124, 5475]
[5974, 5002, 4716, 5827, 5490]
[5094, 3644, 5447, 7895, 5794]
```

例如左上角是 35×8 + 42×18 + 19×6 + 40×6 + 18×7 + 3×42 + 36×33 + 9×41 + 50×40 + 8×25 = 5399。本次使用保存的两个输入矩阵重新核对，25 个结果都一致。

补充方法生成是那天上完课得出的。

3. Pascal triangle：杨辉三角

代码：

[3]:
```
# 3. Pascal triangle

def Pascal_triangle(k):
    row = [1]

    for i in range(1, k):
        new_row = [1]

        for j in range(len(row) - 1):
            new_row.append(row[j] + row[j + 1])

        new_row.append(1)
        row = new_row

    print(row)


Pascal_triangle(100)
Pascal_triangle(200)
```

输出结果：

```
[1, 99, 4851, 156849, 3764376, 71523144, 1120529256, 14887031544, 171200862756, 1731030945644, 15579278510796, 126050526132804, 924370524973896, 6
186171974825304, 3800077070498296, 215337700647490344, 1130522928399324306, 5519611944537877494, 25144898858450330806, 107196674080761936594, 428
786696323047746376, 1613054714739084379224, 5719012170438571889976, 19146258135816088501224, 60629817430084280253876, 18188945229025284076162 8, 51
76853642107196237061 72, 13996678365697234270574 28, 35991458654650030981476 72, 88117019464832834471891 28, 20560637875127661376774632, 4576400043173
5762419272568, 9724850091743849514095420 7, 19744392610510239922557369 3, 38327350361578701026140775 7, 71179364957217587619975726 3, 1265410932572757
113244012912, 21546186149211810306587246 88, 35154303717135058921273929 12, 54984936583211246005069478 88, 82477404874816869007604218 32, 11868699725 8
88281149874753368, 16390109145274293016493707032, 21726423750712434928840495368, 27651812046361280818524266832, 33796659167774898778196326128, 396
74339023040098565708730672, 44739148260023940935799206928, 48467410615025936013782474172, 50445672272782096667406248628, 50445672272782096667406246 24
8628, 48467410615025936013782474172, 44739148260023940935799206928, 39674339023040098565708730672, 33796659167774898778196326128, 2765181204636128
0818524266832, 21726423750712434928840495368, 16390109145274293016493707032, 11868699725888281149874753368, 82477404874816869007604218 32, 54984936
583211246005069478 88, 35154303717135058921273929 12, 21546186149211810306587246 88, 12654109325727571132440129 12, 71179364957217587619975726 3, 38327
350361578701026140775 7, 19744392610510239922557369 3, 97248500917438495140954207, 45764000431737576241927256 8, 20560637875127661376774632, 881170194
6483283447189128, 35991458654650030981476 72, 13996678365697234270574 28, 51768536421071962370617 2, 18188945229025284076162 8, 60629817430084280253 87
6, 19146258135816088501224, 57190121704385718899 76, 16130547147390843792 24, 42878669632304774637 6, 10719667408076193659 4, 25144898858450330806, 55
19611944537877494, 1130522928399324306, 215337700647490344, 38000770702498296, 6186171974825304, 924370524973896, 126050526132804, 15579278510796,
1731030945644, 171200862756, 14887031544, 1120529256, 71523144, 3764376, 156849, 4851, 99, 1]

[1, 199, 19701, 1293699, 63391251, 2472258789, 79936367511, 2203959847089, 52895036330136, 1122550215450664, 21328454093562616, 36646162033484858
4, 574123205191262781 6, 82585414900589338584, 10972062265364012126 16, 13532210127282281622264, 15562041646374623865603 6, 16752080125215036278855 6
4, 16938214348828536681954036, 16135877879673500733861476 4, 14522290091706150660475328 76, 12378523459120956991548018324, 10015350798743319747707 0
30076, 77074656146850765014962819232 4, 56521414507690561010972734103 76, 39564990155383392707680913872632, 26478108796295039735140303899376 8, 1696 5
60304355200694140471323923032, 10421727583896232835434323846955768, 61452255753319166029629978545842632, 34822944926880860750123654509310 8248, 18 5
84121589170533763777089071204933 52, 99666638343145302259829717623825900 98, 50437359403955349931489584373269471102, 24625299003107612025374326488 1 2
56829498, 11609069530036445669105039630 11639339062, 52885761192388252492589624981641347668 38, 23298321822592662584573267221641999107962, 99324424 6
12105561544759718155421154091838, 41003159903971783099246960571853348322765 62, 16401263961588713239698784222874133929106248, 6360490170469769280761 2
35835048470603119352, 23927558260338655865720839569944246554591848, 87363410392399278393445856104215039745835352, 30974300048214289612221712618767 3
1504553416248, 10668925572162699755320812124242018490173226 32, 35717707350283820919987066676810235814927757 68, 11627253669347711916506428088404843 8
467412653032, 36819636619601087735603688946626721813473401268, 11346459448081151526686034757021704069049966513 2, 34039378344243454580058104271065 3
122071498995396, 99448379868475975145659951693896112134614412380 4, 28304538885643162156841678559031970376774874871215 96, 78505041814897072397278631 7
31641442525642654480 4, 21225437231435134388893644487559194557174782880396, 55957970882874445207083244558110603832551700321044, 14389192512739140 5
32499771720855838426561515111256, 36099202268801709765170995361548043675435608177034 4, 88380805554652461838866919678272796584687178640323 6, 21121 5
14547806774778441077414638075116001151218353544, 49283533944882474483029180634155508604003528428249 36, 11230182325145350742854190341225599501568017 2
133650264, 24996212272097716169578681727244076309941715555544136, 54356842559958525638607609470356165943841508430310264, 11550829043991186698204 1 3
70124506852630663205414409311, 23990183399058618527039319948936038623291588816838856 9, 48707342052634164888231346562991351144258680325097073 1, 96 6
```

```
87708850751401942309986460863428390841857958774786 9, 18768790541616448612330762077697018452339890074350399 81, 35633500883358764756743910611279846626
26906168112172498 19, 66176501640523402626252440542094828659282574077974892521, 12023617903700734104036124365214547845738761352940297679, 2137532071
76901939627308877603814183924244646274494180 96, 37187201796529515524203051309156714189560369968302412304, 63318749004901607514183573850726291335 7
57650811635668 96, 10553124834150267919030595641787716188929294180193927 8160, 17218256308350437131049919205022063255621479978211145384 0, 2750448734
97026463262225982106196594862524939911684530160, 43019839187996446817937910021738441760548772652853221384 0, 65891146098070507151525153324434828519
32153786069923781 60, 98836719147105760727287729986652242778982306791048856724 0, 14520456269752998 60760, 208952
90729654608433191422831565353705246455163503583952 40, 29454807414091435984137306883049956427877533182288184607 60, 40675686428983411597141995219449
93982897373629935035017240, 55031811050977556866721522943961683298023290287356351660 11560, 72949144881528389334956437390832929022961105729751448804 4
0, 94750038754169057412069855461656562983846033878872571435 60, 12059095841439698216081617967847198925216767948220145455440, 15039995937076477550393928027315045850551249912948720 73
39280273150458505512499129487207365 60, 18382217256426805894925912033385056039562638782492880900240, 22018260230225514753262905622406275915520083 81
63925716277 60, 25847522878960386884265150078476932596480098393156497128240, 29738547828481305339960979122548728901326564817932744007 60, 33534958 1
89564025170594295606278353867453360326605009200240, 37064953788465501504341063564833970064027398255721325958160, 40153699937504293296369485528570 1
34236029681443698103121340, 42637433954257136180681000097347668312485125656710356922660, 44377737380961509086104918468667981304831457316167922511 3
40, 45274257328051640582702088538742081937252294837706668420660, 45274257328051640582702088538742081937252294837706668420660, 44377737380961509086
01491846866798130483145731616792251134 0, 42637433954257136180681000097347668312485125656710356922660, 40153699937504293296369485528570134236029681
443698103121340, 37064953788465501504341063564833970064027398255721325958160, 33534958189564025170594295606278353867453360326605009200240, 2973854
78284813053399609791225487289013265648179327440077 60, 25847522878960386884265150078476932596480098393156497128240, 22018260230225514753262905622 40
62759155200838163925716277 60, 18382217256426805894925912033385056039562638782492880900240, 15039995937076477550393928027315045850551249912948720 73
```

```
6560, 12059095841439698216081617967847198925216767948220145455440, 94750038754169057412069855461656562983846033878872571435 60, 72949144881528389 33
49564373908329290229611057297514488044 0, 55031811050977556866721522943961683298023290287356351661560, 40675686428983411597141995219449939839828973 62
9935035017240, 29454807414091435984137306883049956427877533182288184607 60, 20895290729654608433191422831565353705246455163503583952 40, 14520456269
75998213153980230668100850703567223226520240760, 98836719147105760727287729986652242778982306791048856724 0, 65891146098070507151525153324434828519
32153786069923781 60, 43019839187996446817937910021738441760548772652853221384 0, 27504487349702646326222598210619659486252493991168453016 0, 1721825
63083504371310499192050220632556214799782111453840, 10553124834150267919030595641787716188929294180193927 8160, 63318749004901607514183573850726297
13357576508116356689 6, 37187201796529515524203051309156714189560369968302412304, 21375320717690193962730887760381418392424464627449418096, 1202361
79037007341040361243652145478457387613529402976 79, 66176501640523402626252440542094828659282574077974892521, 35633500883358764756743910611279846626
90616811217249819, 18768790541616448612330762077697018452339890074350399 81, 96687708850751401942309986460863428390841857958774786 9, 48707342052634
16488823134656299135114425868032509707 31, 23990183399058618527039319948936038623291588816838856 9, 11550829043991186698204117012450685263066320541 4
409311, 54356842559958525638607609470356165943841508430310264, 24996212272097716169578681727244076309941715555544136, 11230182325145350742854190 34
12255995015680171336502 64, 49283533944882474483029180634155508604003528428249 36, 21121514547806774778441077414638075116001512183535 44, 88380805554
65246183886691967827279658468717864032 56. 36099202268801709765170995361548043675435608177034 4. 14389192512739143053249977172085583842656151511125
```

(1)第一种方法：从上一行生成下一行

程序把 [1] 作为第 1 行。每次生成新的一行时，先放入开头的 1，再把上一行相邻的两个元素相加，最后补上末尾的 1。例如从 [1, 2, 1] 得到 [1, 3, 3, 1]。

```
row = [1]
for i in range(1, k):
    new_row = [1]
    for j in range(len(row) - 1):
        new_row.append(row[j] + row[j + 1])
    new_row.append(1)
    row = new_row
```

range(1, k) 一共执行 k−1 次，因为最初已经有了第 1 行。row = new_row 表示本轮得到的新行会作为下一轮的输入。程序没有保存整个三角形，只保留当前的一行。

(2)第二种方法：用组合数直接计算

```
from math import comb
row = [int(comb(k - 1, i)) for i in range(k)]
```

按“[1] 是第 1 行”的约定，第 k 行第 i+1 个元素为 C(k−1, i)。列表推导式让 i 从 0 到 k−1，逐个计算后组成列表。comb 已经返回整数，因此外面的 int 可以省略；这个方法也没有实际使用 NumPy。本次逐项比较了两种方法生成的第 100 行和第 200 行，结果完全相同。还可以用每行元素之和等于 2^(k−1) 来检查。Python 的普通整数能保存这些很大的整数，不需要改为浮点数。

原题所链接的 Math is Fun 页面明确把顶端 [1] 标为 Row 0 [2]，因此当前两次调用分别对应该网页的 Row 99 和 Row 199。若按网页编号计算 Row 100 和 Row 200，应分别有 101 和 201 个元素。作业正文使用“第 k 行”的表述，没有另外定义起始编号；报告应明确采用的约定，不能把两种编号混用。当前稿如实保留原代码输出，没有把它改写成另一套编号的结果。另需注意，两个函数同名，后定义的版本会覆盖前一个。

4. Add or double：最少操作次数

代码及结果：

[4]:
```
# 4. Add or double
import random
#定义函数，输入目标金额 x，初始操作次数为 0，
#只要 x 大于 1，就继续计算，判断 x 是否为偶数，如果是偶数，就除以 2，奇数...
def Least_moves(x):
    moves = 0
    while x > 1:
        if x % 2 == 0:
            x = x // 2
        else:
            x = x - 1
        moves = moves + 1
    print(moves)
Least_moves(2)
Least_moves(5)

for i in range(5):
    x = random.randint(1, 100)
    print("x =", x)
    Least_moves(x)
```

```
1
3
x = 23
7
x = 28
6
x = 41
7
x = 88
8
x = 9
4
```

按照代码的含义，起点是 1，每一步可以加 1 或乘以 2，目标是正整数 x。Least_moves(x) 从目标倒着算：遇到偶数就除以 2，遇到奇数就减 1，直到回到 1。

```
moves = 0
while x > 1:
    if x % 2 == 0:
        x = x // 2
    else:
        x = x - 1
    moves = moves + 1
```

x % 2 用来判断奇偶，余数为 0 表示偶数。x // 2 是整数除法。moves 每轮增加 1，记录倒推的次数；倒推一步对应正向操作的一步。

(1)为什么可以这样倒推

奇数不能由整数翻倍得到，所以它的最后一步只能是加 1，倒推时减 1 是必需的。对于大于 2 的偶数，如果倒推选择减 1，得到的奇数还要再减 1，两步才减少 2。除以 2 可以更快接近起点。

更具体地说，任何正向方案都可以按翻倍操作分成几段加一。如果某次翻倍之后连续加了两次 1，把这两次加一换成翻倍前加一次 1，最终数值相同，操作却少一次。因此最优方案在每次翻倍后至多加一次 1；偶数目标可以选择以翻倍结束，奇数目标则必须以加一结束。这正好对应代码的倒推规则。本次还核对了 1—100 的最优次数，结果一致。

(2)示例结果

x=2 时只需 1→2，共 1 次。x=5 可以走 1→2→4→5，共 3 次，与保存的输出一致。

保存的随机测试分别为：23 需要 7 次，28 需要 6 次，41 需要 7 次，88 需要 8 次，9 需要 4 次。例如 23 的倒推过程是 23→22→11→10→5→4→2→1，共 7 次。

函数只输出次数，没有输出操作路径，也没有返回 moves。它没有检查输入范围：输入 0 或负数会打印 0，不能理解为正确解决了这类输入。上述分析以 x 是正整数、起点为 1、只允许加 1 和翻倍为前提。

5.1 数字表达式：寻找指定目标

代码及结果：

[5]:
```python
#5.1 Dynamic programming
import itertools
import random

def Find_expression(x):
    digits = "123456789"
    count = 0

    for ops in itertools.product(["+", "-", ""], repeat=8):
        expression = digits[0]

        for i in range(8):
            expression = expression + ops[i] + digits[i + 1]

        if eval(expression) == x:
            print(expression, "=", x)
            count = count + 1

    return count


x = random.randint(1, 100)
print("Target =", x)
print("Total solutions =", Find_expression(x))
```

```
Target = 2
1+2+34-5-6-7-8-9 = 2
1+23+4+56+7-89 = 2
1-2+34+56-78-9 = 2
12+3+4+5+67-89 = 2
12+3+4-5-6-7-8+9 = 2
12+3-4+5-6-7+8-9 = 2
12+3-4-5+6+7-8-9 = 2
12-3+4+5-6+7-8-9 = 2
12-3-4-5-6+7-8+9 = 2
123+4-56-78+9 = 2
123-45+6+7-89 = 2
Total solutions = 11
```

Find_expression(x) 保持数字 1—9 的顺序不变，在相邻数字之间加入加号、减号，或者不加符号。不加符号表示拼接成多位整数，例如 1 和 2 拼接得到 12。这里不使用乘号、除号、括号，也不允许在开头添加负号。

(1)枚举过程

```
    for ops in itertools.product(["+", "-", ""], repeat=8):
        expression = digits[0]
        for i in range(8):
            expression = expression + ops[i] + digits[i + 1]
        if eval(expression) == x:
            print(expression, "=", x)
            count = count + 1
```

9 个数字之间有 8 个空位，每个空位有 3 种选择，因此一共有 3^8 = 6561 种表达式。itertools.product 逐个给出这 8 个空位的选择，内层循环负责把符号和数字接成字符串。eval(expression) 计算字符串表示的算式。如果结果等于 x，程序就打印这个表达式，并让 count 增加 1。函数最后返回 count，外面的 print 再显示解的总数。

notebook 的这次随机目标是 2，程序找到了 11 个表达式。下面列出其中 3 个，完整结果在原文件中。

```
1+2+34-5-6-7-8-9 = 2
12+3+4+5+67-89 = 2
123-45+6+7-89 = 2
Total solutions = 11
```

例如最后一个表达式是 123−45+6+7−89，依次计算得到 78、84、91、2。本次重新枚举也得到 11 个解。统计的是符合要求的表达式个数，而不是不同的计算结果个数。

5.2 统计 1—100 的解数与代码检查

代码：

[6]:
```
#5.2 Dynamic programming
import matplotlib.pyplot as plt

Total_solutions = []

for x in range(1, 101):
    count = 0

    for ops in itertools.product(["+", "-", ""], repeat=8):
        expression = "1"

        for i in range(8):
            expression = expression + ops[i] + str(i + 2)

        if eval(expression) == x:
            count = count + 1

    Total_solutions.append(count)

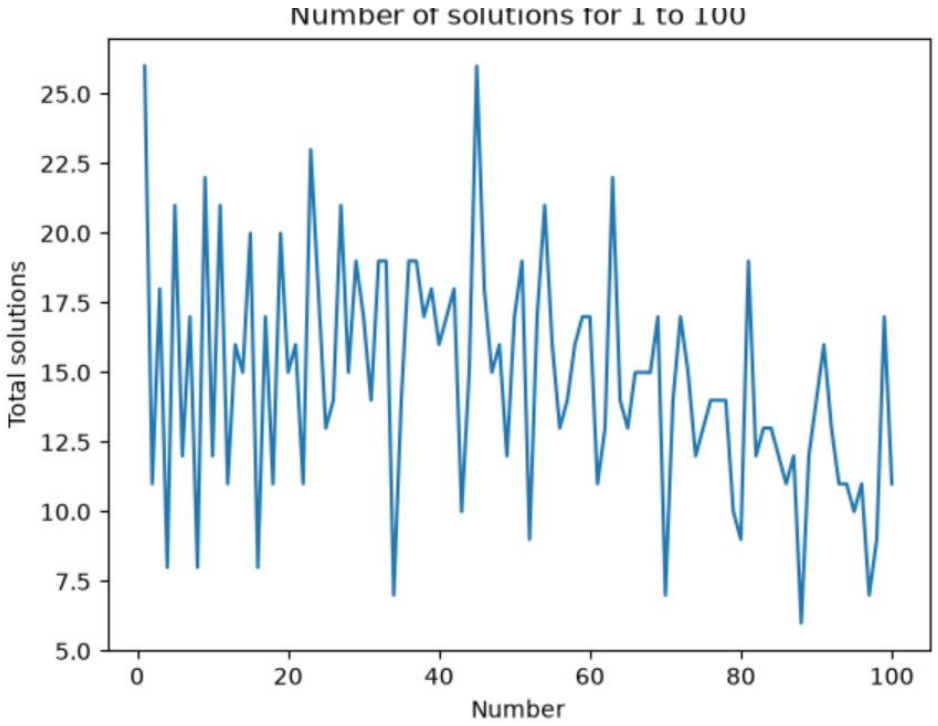
plt.plot(range(1, 101), Total_solutions)
plt.xlabel("Number")
plt.ylabel("Total solutions")
plt.title("Number of solutions for 1 to 100")
plt.show()

maximum = max(Total_solutions)
minimum = min(Total_solutions)

print("Maximum solutions:", maximum)
print("Numbers with maximum solutions:")
for i in range(100):
    if Total_solutions[i] == maximum:
        print(i + 1)

print("Minimum solutions:", minimum)
print("Numbers with minimum solutions:")
for i in range(100):
    if Total_solutions[i] == minimum:
print(i + 1)
```

输出结果：

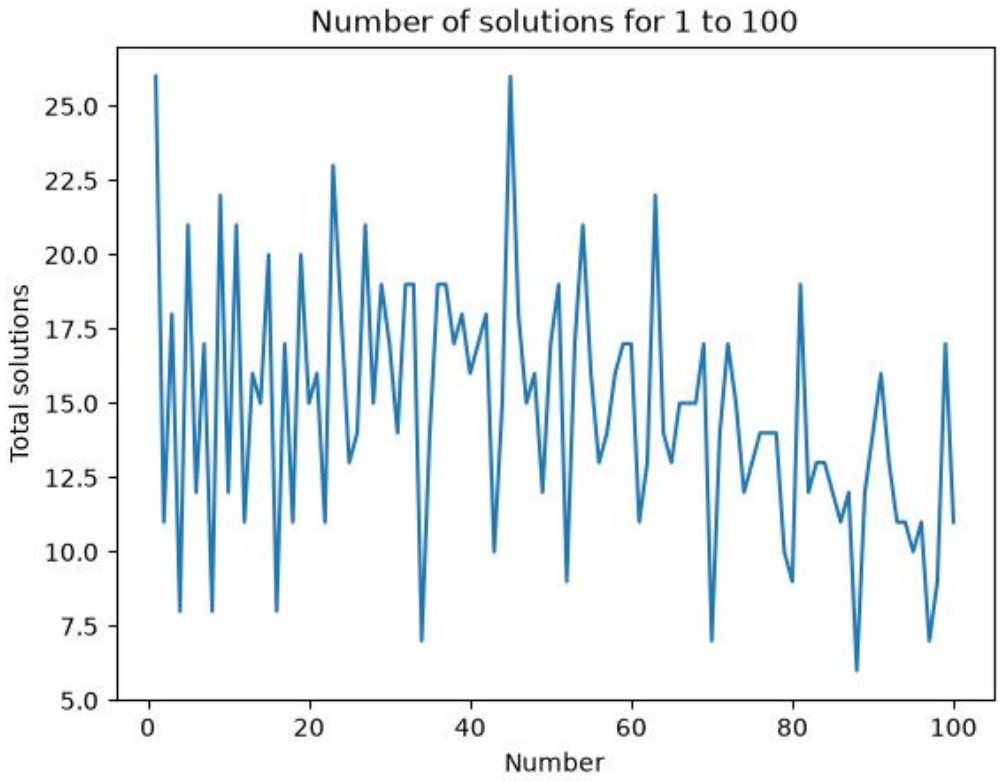



```
Maximum solutions: 26
Numbers with maximum solutions:
1
45
Minimum solutions: 6
Numbers with minimum solutions:
88
```

第二部分依次取 x=1、2、…、100。对每个目标重新枚举 6561 个表达式，记录满足条件的数量，再把结果放入 Total_solutions。列表第 0 项对应目标 1，所以输出对应目标时需要用 i+1。

(1)数量与图形的含义

横轴是目标值，纵轴是满足条件的表达式数量。曲线表示各个目标的解数差异，不表示程序运行速度。目标增加时解数没有单调增减关系。保存的输出为：最多 26 个解，对应目标 1 和 45；最少 6 个解，对应目标 88。本次独立枚举验证了这些数值。统计范围仅限 1—100，不能把它解释为所有可能结果的全局极值。



图：原 notebook 保存的 1—100 目标值对应解数。

(2)目前的运行问题

最后一行 print(i + 1) 没有缩进到 if 下面。当前源码会在执行该单元格之前报 IndentationError，因此连前面的绘图也不会执行。文件中有保存的图和结果，只能说明 notebook 保留过这些输出，不能证明它们来自当前这份有缩进错误的源码。

此外，第 5.2 部分依赖第 5.1 部分导入的 itertools。重新启动内核后，单独运行第 5.2 部分也会缺少这个名称。